

EXERCISE 19 · UNIT 3

Shared-memory tiling for matrix multiplication

The set piece of GPU programming. Stage tiles in shared memory so every loaded element is reused, then measure how much that is actually worth on hardware built this decade.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

EXERCISE

ex19-cuda-tiled-matmul

LANGUAGE

CUDA C++

SUBMIT AS

<ROLLNO>.cu

UNIT

3 — GPU and CUDA

MARKS

12

OFFERING

Odd Semester 2026-27

The problem

Compute $\mathbf{C} = \mathbf{A} \cdot \mathbf{B}$ on the GPU, where $\mathbf{A}$ is $m \times k$, $\mathbf{B}$ is $k \times n$, all row-major.

`gpu_matmul_naive` is written for you and works. Every thread computes one output element by reading a whole row of $\mathbf{A}$ and a whole column of $\mathbf{B}$ straight from global memory. Each element that arrives is used exactly once, giving an arithmetic intensity of roughly 0.25 FLOP per byte.

You implement `gpu_matmul_tiled`. Each block cooperatively loads a `TILE` $\times$ `TILE` tile of $\mathbf{A}$ and of $\mathbf{B}$ into **shared memory**, then every thread in the block uses those tiles `TILE` times before they are replaced.

The algorithm

```
acc = 0
for each tile t along the k dimension:
    As[ty][tx] = A[row][t*TILE + tx]      (0 if off the edge)
    Bs[ty][tx] = B[t*TILE + ty][col]      (0 if off the edge)
    __syncthreads()
    for i in 0..TILE-1:
        acc += As[ty][i] * Bs[i][tx]
    __syncthreads()
C[row][col] = acc                        (if inside the matrix)
```

PITFALL · BOTH `__syncthreads()` ARE REQUIRED, AND THEY GUARD DIFFERENT THINGS

The **first** ensures every thread has finished *writing* its tile element before any thread *reads* the tile. The **second** ensures every thread has finished *reading* the tile before the next iteration *overwrites* it.

Remove either and you get a race. The answer will be correct most of the time, and wrong more often as the matrices grow. One of the hidden tests runs the same multiplication five times and compares the results to each other, which is how a missing barrier is caught.

PITFALL · NO THREAD MAY RETURN EARLY

`__syncthreads()` must be reached by **every** thread in the block. A tempting optimisation is to return immediately when `row >= m || col >= n`, but those threads still have to participate in the barriers and still have to help load tiles. Zero-pad instead of returning.

Handling sizes that are not multiples of TILE

Real matrices are not conveniently sized. Your kernel is tested with m , k and n prime, with $k = 1$, with $1 \times 250 \times 250$, and with $250 \times 250 \times 1$.

When a tile load runs off the edge of a matrix, write `0.0f` into shared memory rather than skipping the load. Zeros contribute nothing to the dot product, so the arithmetic stays correct and every thread still reaches both barriers.

What tiling is actually worth

This is the part where the textbooks will mislead you.

The classic claim is that tiling gives roughly a ten-fold speedup. On one RTX 6000 Ada of `asaicompute`, 4096×4096, best of three runs, the measurement is:

VERSION	TIME	GFLOP/S	SPEEDUP
Naive	24.42 ms	5 628	1.00x
Tiled, TILE = 32	19.44 ms	7 071	1.26x
cuBLAS <code>SGEMM</code>	2.17 ms	63 302	11.25x

BEYOND THE SYLLABUS · WHY THE TEXTBOOK NUMBER IS WRONG HERE

Two things changed since those books were written.

The L2 cache got enormous. Most of the re-reads the naive kernel performs are served from L2, never reaching DRAM. Tiling is saving traffic that had already stopped being expensive.

The naive kernel is already coalesced. Within a warp every thread shares the same `row`, so `A[row*k+t]` is a single broadcast address, and `col` is consecutive, so `B[t*n+col]` is a perfectly coalesced read. The naive kernel is not the disaster its reputation suggests.

Tiling is still correct practice, and the technique generalises to problems where cache cannot rescue you. But this is a lesson in **measuring on the machine you actually have** rather than quoting a number from a book.

The cuBLAS row is the real lesson: a tuned library is about **nine times faster than your best hand-written tiled kernel**, because it also does register blocking and software pipelining, and on this hardware uses TF32 tensor cores for single precision by default. Note that this means cuBLAS is not doing quite the same arithmetic your kernel is, which is exactly the kind of caveat that belongs in any performance comparison you report.

Building and running

```
./selfcheck.sh          # compiles and runs the public tests via srun
./selfcheck.sh AIE23001.cu # test a specific file
```

By hand, on the cluster:

```
export PATH=/usr/local/cuda/bin:$PATH
nvcc -O2 -std=c++17 -arch=sm_80 -Xcompiler "-Wall,-Wextra,-Werror" \
      -Iinclude -I../harness matmul.cu tests/public.cu -o t -lm
srun --gres=gpu:1 --time=00:10:00 --mem=16G ./t
```

ON ASAI COMPUTE · NEVER RUN THIS ON THE LOGIN NODE

Compiling on the head node is fine. Running compute on it is not: it is shared by everyone, and it has no GPU allocation of its own. Always go through `srun` or `sbatch`.

`-arch=sm_80` targets the Ampere instruction set, which runs on both the A100 nodes and the RTX 6000 Ada cards, so one binary works on any node you land on.

How this is marked

CHECK	MARKS	WHAT IT MEANS
Compiles cleanly	1	Builds with warnings as errors
Uses shared memory correctly	2	<code>__shared__</code> tiles and <code>__syncthreads()</code> present
Public tests	3	Correctness on square, ragged and tiny inputs
Hidden tests	4	Edge sizes, non-square, large, and a repeatability check
Beats the naive kernel	2	At least 1.15x at 8192, and agreeing with the naive result

The speedup threshold is set from measurement on this hardware, not from a textbook. A correct tiled kernel measures about 1.30x at this size, so 1.15x leaves room for run-to-run variation without letting an untiled answer through.

Submitting

Rename your finished file to your roll number:

AIE23001.cu

NOTE · BEFORE YOU SUBMIT

Run `./selfcheck.sh` at least twice. If the two runs disagree on any result, you have a missing or misplaced `__syncthreads()`, and the hidden repeatability test will find it.